From breach to verified repair

One recorded end-to-end journey, backed by frozen evidence and fresh recovery data.

BEFORE · REVISION 7

Seconds

Contract requires milliseconds

100/100 accepted by integer type check

100/100 violate the recorded contract

AFTER · REVISION 8

Milliseconds

Producer configuration repaired through Git

200/200 pass across two recovery windows

Incident resolved after verification

The recorded journey

01 **Commit the fault**

Git-backed revision 7 changes the producer to seconds.

02 **Capture the breach**

Freeze the events, contract and configuration history.

03 **Investigate with evidence**

The agent reads typed tools and cites the affected revision.

04 **Open and review the repair**

PR #1 restores the required unit; an engineer merges it.

05 **Verify fresh data**

Two separate ten-second windows pass before resolution.

Fault commit 1d9e881c572f

Repair PR github.com/dirdr/choker2kafka/pull/1

Measured evidence

A timestamp type-check baseline, one real investigation, and recovery checked against the contract.

What the comparison shows

Check	Integer type baseline	Sentinel
Timestamp type	100/100 accepted	Accepted by ingestion
Meaning of the value	Not evaluated	100% contract failures
Producer attribution	Not evaluated	Evidence-linked revision 7
Repair and outcome	Not evaluated	PR #1 + 2 verified windows

Recorded timings and usage

Detection

11.7s

From faulty revision becoming active

Investigation

31.5s

2 model requests · 3 tool calls

Reported usage

14,829 tokens

claude-sonnet-5-5

Verified recovery

34.7s

From repaired revision becoming active

Data layer and verification

Aiven Kafka carries the events. Aiven PostgreSQL stores contracts, producer revisions, frozen incident evidence, tool activity and recovery records. The agent reads the captured snapshot through typed tools.

Scope: one synthetic end-to-end run, not a production benchmark or an average latency claim. Detection is deterministic; the agent contributes diagnosis and evidence references. The PR adapter constrains the edit. Fresh data establishes recovery.

Source: deployed application APIs. Screenshot capture requires the separate Playwright exporter.